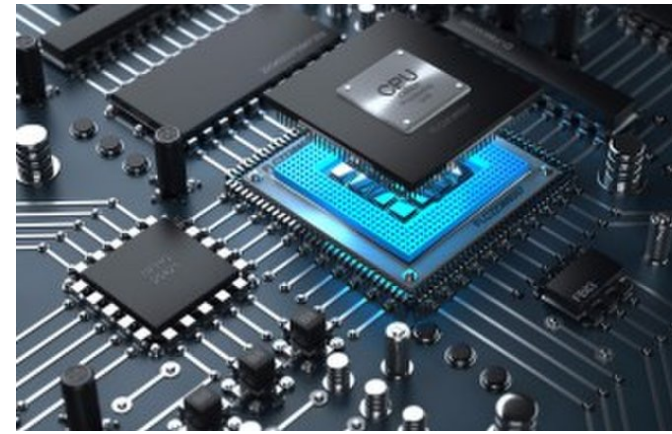


```
loop: lw    $t3, 0($t0)
      lw    $t4, 4($t0)
      add   $t2, $t3, $t4
      sw    $t2, 8($t0)
      addi  $t0, $t0, 4
      addi  $t1, $t1, -1
      bgtz  $t1, loop
```

Assembler

```
0x8d0b0000
0x8d0c0004
0x016c5020
0xad0a0008
0x21080004
0x2129ffff
0x1d20fff9
```



Lecture 05: Assembly Programming Foundations MIPS Instructions

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

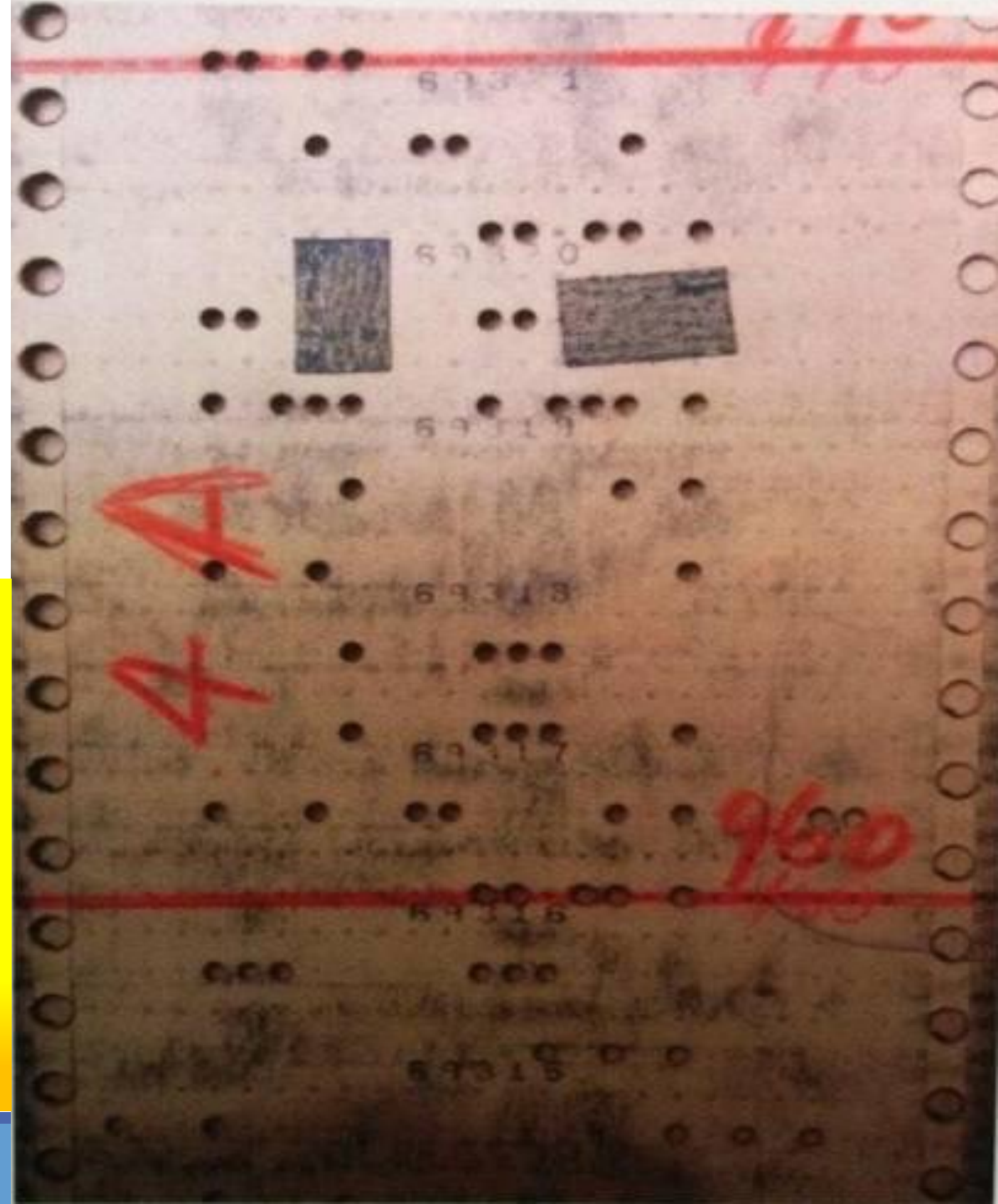
UNIVERSITY OF CALIFORNIA, SANTA BARBARA

This Week on “Didja Know Dat?!”



The term "**patch**", meaning a **software update**, originates from the days when computer code was written on physical cards with holes punched in them (hole? It's a **0**... No hole? It's a **1**).

If there was a change in the code, you would "patch" that section of the card with a piece of tape, thus turning a binary 0 into a 1 (or vice-versa)!!



Administrative

Current homework due TODAY!

New homework will be out on Canvas soon!

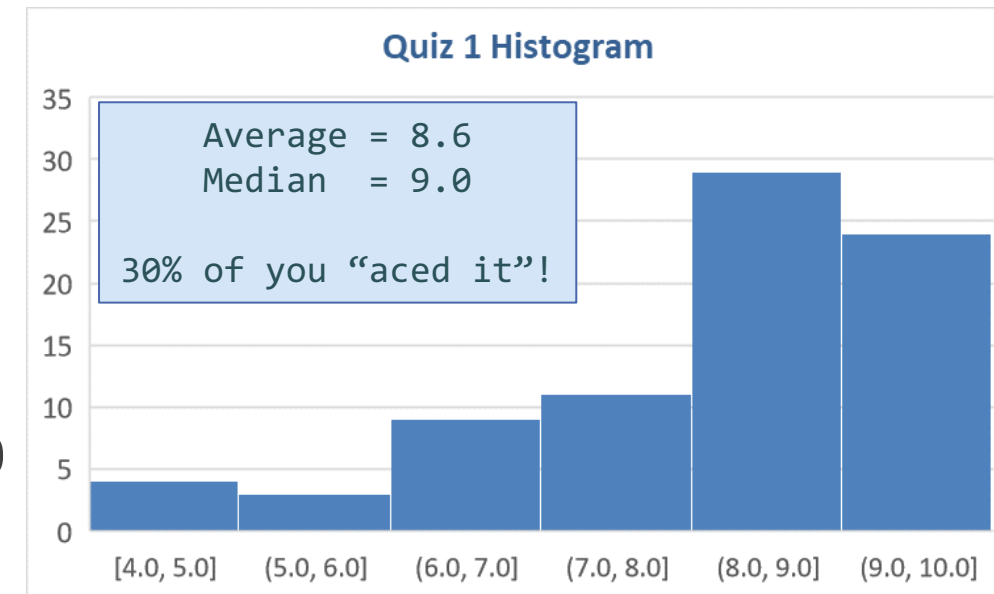
Reading: Handout #2 (still)

Worksheet for this lecture is available on Canvas!

Come to our office hours!! 😊

Quiz #1 is Graded!

- Decided to make Q6 5 pts, so entire quiz is out of 10
- Grades will be posted on Canvas right after class
- Reviewing your quiz – come to Prof. Matni's office hrs (or make an appointment)



DEMO!

`simpleAdd.asm`

The (Pseudo)Instruction **li**

So far, when we've wanted to *assign a value* to a register, we've done this:

```
addi $t0, $zero, 42
```

← 16-bit signed

Technically: this adds the int value of register \$zero (which is 0) to the “immediate” int value of 42 and places the result in register \$t0

There is a “*short cut*” to this using an instruction that **spim** allows, but that is *not really* a core MIPS instruction

```
li $t0, 42
```

← 32-bit signed

- **li** is called a pseudoinstruction – more on these later

System Calls (`syscall`)

To "call over" the Operating System during a program in order to:

- Print some value on the *standard output* (i.e., the display)
- Get user input from the *standard input* (i.e., the keyboard)
- End the program

etc...

In the **spim** emulator, you have to do **2** things in order to do a system call:

1. Set up a ***specific code*** in a ***specific register*** (or two)
 - Usually, the registers that are involved are: `$v0` (always) and `$a0` (sometimes)
2. Use the **spim** command "**syscall**"

Common Examples

Print the integer value that's in register \$t3 (for example)

```
li $v0, 1  
move $a0, $t3  
syscall
```

Get a user integer value input (always ends up in reg. \$v0)

```
li $v0, 5  
syscall
```

Exit a program (always use at the *end* of a program)

```
li $v0, 10  
syscall
```

iClicker!

The **syscall** interaction setup requires you to place a *syscall call code* in _____ and an *optional argument* in _____

- A. register **\$v0** ; register **\$a0**
- B. register **\$a0** ; register **\$v0**
- C. register **\$v0** ; register **\$t0**
- D. register **\$t0** ; register **\$a0**
- E. None of the above

iClicker!!

The *syscall call code* (i.e., that gets put in reg. \$v0) for *printing* an integer to **std.out** is _____, and the one used for *quitting* a program is _____.

A. 1 ; 4

B. 1 ; 10

C. 5 ; 10

D. 1 ; 5

E. None of the above.

iClicker!!!

Consider this MIPS assembly program.

What is missing from it that would prevent it from running on **spim**?

Choose as many correct answers as needed.

- A. The syscall for quitting the program is missing.
- ☒ B. The **main:** label is missing.
- C. The first **addi** instruction is used incorrectly.
- D. The first **li** instruction is not useful
- E. Nothing is wrong!

```
.text
    addi $t0, $zero, 5
    addi $t1, $zero, 7
    add $a0, $t0, $t1
    li $v0, 1
    syscall
    li $v0, 10
    syscall
```

MIPS Instructions... So Far...!

add rd, rs, rt

- Adds the values in registers **rs** and **rt**
and places the result in a destination register (**rd**)
- This syntax is typical of R-Types of MIPS instructions

addi rd, rs, immed

- Adds the value in register **rs** with an “**immediate**” *16-bit signed value*
and places the result in a destination register (**rd**)
- This syntax is typical of I-Types of MIPS instructions

R-Type Syntax

<op> <rd>, <rs>, <rt>

op : operation

rd : register destination

rs : register source

rt : register target

Examples of R-Type CORE Instructions

sub rd, rs, rt (R-Type)

- Subtracts the values in 2 registers (**rs** – **rt**) and places the result in a destination register (**rd**)
- Example: `sub $t0, $t1, $t2` # $\$t0 \leftarrow \$t1 - \$t2$

Examples of R-Type CORE Instructions

sub rd, rs, rt (R-Type)

and rd, rs, rt (R-Type)

- Bitwise ANDs the values in 2 registers (**rs** & **rt**) and places the result in a destination register (**rd**)
- Example: **and \$t0, \$t1, \$t2** # \$t0 ← \$t1 & \$t2

Note the similarity in the syntax!

Same syntax applies with **or** and **nor** instructions (also both R-Types)

Examples of R-Type CORE Instructions

sub rd, rs, rt (R-Type)

and rd, rs, rt (R-Type)

sll rd, rt, shamt (R-Type)

- Shifts an amount of bits (**shamt**) to the *left* in the value in register **rt** and places the result in a destination register (**rd**)
- Example: `sll $t1, $t0, 3` # $\$t1 \leftarrow \$t0 \ll 3$
 # i.e., mult. value in $\$t0$ by 2^3 & put in $\$t1$
- The **shamt** value is restricted to be a positive number in 5-bits – *why do you think it is so?*

MIPS Peculiarity: NOR used as NOT

*MIPS does not have a **logic NOT** instruction!* (there's a good reason for it... which is...?)

How can we make a NOT function using **NOR** instead (which MIPS *does* have)

Recall: NOR = NOT OR

Truth-Table:

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

Note that:
 $0 \text{ NOR } \mathcal{B} = \text{NOT } \mathcal{B}$

So, in the absence of a NOT function, use a NOR with a 0 as one of the inputs!!

Example: `nor $t1, $t0, $zero` will do a **bit-wise NOT** on `$t0` and put the result in `$t1`

I-Type Syntax

<op> <rt>, <rs>, immedi

op : operation

rt : register target (acts as a destination in most I-types)

rs : register source

immedi: immediate 16-bit signed integer value

Examples of I-Type CORE Instructions

andi *rt, rs, immediate* (I-Type)

- Bitwise ANDs the values in register **rs** & **immediate value** and places result in destination register (**rd**)
- Examples: `andi $t0, $t1, 0xF` # $\$t0 \leftarrow \$t1 \& 0x000F$
`andi $a0, $v0, 17` # $\$a0 \leftarrow \$v0 \& 0x0011$

Same syntax applies with **ori** instruction (also I-Type)

NOT ALL R-Type instructions have I-Type equivalents!!!

e.g., there are NO core instructions **subi** or **nori**

(don't use these in our course!!!!!!!!!!!!!!)

Examples of I-Type CORE Instructions

andi rt, rs, immediate (I-Type)

lw rt, offset(rs) (I-Type)

- Load a “word” (a 32-bit value) *from memory* which is at address value **rs** + **offset**
(offset is always an immediate value)
- Example: **lw \$t1, 4(\$t0)**
Get the “word” (32-bit value) from memory at address (\$t0 + 4)
and place that in register \$t1 (that is, dereference a ptr)
i.e., $\$t1 \leftarrow *(\$t0 + 4)$

I-Type CORE Instructions: Branching

`beq rt, rs, label` (I-Type)

- **Branch-on-equal**: If the values in `rs` and `rt` are equal, go to the part of the program where the **label** is
- Example: `beq $t0, $t1, here` # If `$t0 = $t1`, go to label “**here**”

main:

...

`beq $t0, $t1, here`

`addi $t0, $t0, 1`

`addi $t1, $t1, -1`

Execute these instructions IF `$t0 != $t1`

Skip them if `$t0 == $t1`

here:

`# other stuff done here`

Execute these instructions IF `$t0 == $t1`

I-Type CORE Instructions: Branching

beq rt, rs, label (I-Type)

bne rt, rs, label (I-Type)

- Branch-on-NOT-equal:

If the values in **rs** and **rt** are NOT equal, go to the part of the program where the **label** is

...

bne \$t0, \$t1, there

addi \$t0, \$t0, 1

addi \$t1, \$t1, -1

} *Execute these instructions IF \$t0 == \$t1*
Skip them if \$t0 != \$t1

there:

stuff done here

} *Execute these instructions IF \$t0 != \$t1*

I-Type CORE Instructions: Branching

beq rt, rs, label (I-Type)

bne rt, rs, label (I-Type)

MIPS also has other core branching instructions but we will not be using them in this course (and you should avoid using them)

J-Type Instructions

J-Type instructions are used to “**jump**” from one place in the program to another.

There are only 2 of these in MIPS:

j “jump” *simple jump: go to an instruction, do not return (used in if/else, loops)*

jal “jump-and-link” *jump but remember how to come back (used for functions)*

Pseudoinstructions

Introducing **pseudoinstructions** to make some programming a little easier

- These are **spim** “macros” – they’re not CORE instructions in MIPS
- In other words, they are provided by the EMULATOR but they are **not actually** hardware implementations!
- In CS64, we are allowed to use only a selected and limited number of them (details to follow)

EXAMPLE:

li reg, value

example: **li \$t0, 5**

- **li** = “load immediate” - Places a 32-bit signed value in a register
- This instruction is actually a “macro” made up of 2 core (i.e. “regular”) instructions

li \$t0, 0x1234ABCD # 32b no.

Is a macro for:

lui \$t0, 0x1234

ori \$t0, \$t0, 0xABCD

List of all PsuedoInstructions in MIPS

That You Are Allowed to Use in CS64!!!

PSEUDOINSTRUCTION SET

NAME	MNEMONIC
Branch Less Than	blt
Branch Greater Than	bgt
Branch Less Than or Equal	ble
Branch Greater Than or Equal	bge
Load Immediate	li
Move	move
Load Address	la

EXAMPLE USES:

```
blt $t0, $a0, label    # branch if t0 < a0
bgt $t0, $a0, label    # branch if t0 > a0
ble $t0, $a0, label    # branch if t0 <= a0
bge $t0, $a0, label    # branch if t0 >= a0
li $t0, -1123          # assign this 32b int to t0
move $t0, $t3          # copy value from t3 to t0
la $t0, mem_add_label  # copy mem address (label) to t0
```

You may find others (e.g., online), but these are the only pseudoinstructions that you are allowed to use in this course

More on Pseudoinstructions

Examples of pseudoinstructions that are *commonly found in online solutions* (which is why I'm mentioning them) **and that I do not allow in CS 64:**

Inappropriate Syntax Shortcuts that are not actually in the hardware:

```
addi $t0, 9           # supposed to be short-hand for addi $t0, $t0, 9
beq $t0, 5, label     # beq/bne must have the syntax: $rs, $rt, label
```

Not actual hardware instructions:

```
subi $t0, $t1, 3      # subi is not a core instruction!
mul $t0, $t1, $t2     # mul is not a core instruction!
```

.text

main:

li \$t0, 5 # Instead of: addi \$t0, \$zero, 5

li \$t1, 7 # Instead of: addi \$t1, \$zero, 7

add \$t2, \$t0, \$t1 # ALTERNATIVE!!! add \$a0, \$t0, \$t1

Print the integer that's in \$t2 to std.output - MORE ON HOW THIS WORKS LATER!

li \$v0, 1

move \$a0, \$t2

ALTERNATIVE!!! Drop the move instruction if you did the above

syscall

End program - MORE ON HOW THIS WORKS LATER!

li \$v0, 10

syscall

To Dos



Keep up with the reading

Work on your new assignment

Watch the syscall video if you haven't done so yet!

Come to office hours!! 😊

Don't forget Lab on Friday!

</LECTURE>